

Nama : Hasan Zubeir Pohan

NIM : 714240045

Kelas : 3C

Matkul : Pemrograman 4

JURNAL PRAKTIKUM PEMROGRAMAN IV

Modul 03 – Null Safety, Error Handling, Async/Await, dan Future.wait

1. Tujuan Praktikum

Mahasiswa mampu menerapkan *null safety*, penanganan *error*, serta pemrograman *asynchronous* pada Dart untuk memantau status kiriman pada proyek SiLog. Secara khusus, praktikum ini bertujuan untuk:

- Menggunakan tipe *nullable* (*String?*) pada properti opsional dan memahami perbedaannya dengan tipe *non-nullable*.
- Menyusun *exception* khusus *ResiTidakDitemukan* dan menanganinya dengan struktur `try / on / catch / finally`.
- Menggunakan `Future`, `async`, dan `await` untuk mengambil data yang membutuhkan waktu tunggu.
- Menggunakan `Future.wait` agar beberapa operasi berjalan secara paralel, serta membandingkan waktunya dengan proses sekuensial.
- Menerapkan seluruh konsep tersebut pada Tugas Praktikum, yaitu memantau banyak resi sekaligus dengan fungsi `pantauBanyakResi` yang tetap berjalan meskipun salah satu resi gagal.

2. Alat dan Bahan

No	Alat / Bahan	Keterangan
1	Flutter SDK & Dart SDK	Flutter 3.47.4 (channel stable); Dart SDK bawaan Flutter (terpasang pada Modul 1)
2	Visual Studio Code	Dengan ekstensi Flutter dan Dart
3	Terminal PowerShell	Menjalankan <code>dart run</code> dan <code>dart analyze</code>
4	Git & GitHub	https://github.com/Nogi3201/silog-714240045/tree/main/Pertemuan%203
5	Proyek	<code>silog_app</code> (proyek yang sama dari modul sebelumnya)

3. Langkah Kerja

A. Penyiapan Berkas Latihan

Proyek `silog_app` dibuka melalui *File > Open Folder* pada Visual Studio Code, kemudian dibuka terminal terintegrasi. Pada direktori latihan dibuat dua berkas untuk Modul 03, yaitu `modul03_nullsafety.dart` dan `modul03_async.dart`.

B. Latihan Null Safety

Kode diketik pada `latihan/modul03_nullsafety.dart` dan dijalankan dengan `dart run latihan/modul03_nullsafety.dart`. Properti catatan bertipe `String?` kemudian diubah sementara menjadi `String`, lalu diperiksa dengan `dart analyze latihan/modul03_nullsafety.dart` untuk mengamati reaksi *analyzer*. Setelah pengujian, kode dikembalikan menjadi `String?` catatan.

C. Penanganan Error / Exception

Pada berkas yang sama, pencarian resi dibungkus dengan `try`. *Exception* khusus `ResiTidakDitemukan` ditangkap dengan klausa `on`, sedangkan blok `finally` digunakan untuk pesan penutup pencarian. Pengujian dilakukan dengan resi yang terdaftar (SLG-001) dan resi yang tidak terdaftar (SLG-999).

D. Latihan Async/Await

Kode diketik pada `latihan/modul03_async.dart` dan dijalankan dengan `dart run latihan/modul03_async.dart`. Status kiriman dan ongkos kirim resi SLG-002 diambil melalui fungsi *asynchronous* menggunakan `await`. Sebagai eksperimen, kata kunci `await` dihapus sementara pada pemanggilan fungsi untuk mengamati hasilnya, kemudian dikembalikan seperti semula.

E. Latihan Future.wait

Status dan ongkos kirim resi SLG-001 diambil dengan dua cara: secara sekuensial (satu per satu dengan `await`) dan secara paralel (dengan `Future.wait`). Durasi masing-masing proses diukur dan ditampilkan dalam milidetik untuk dibandingkan.

F. Tugas Praktikum – Pantau Banyak Resi

Fungsi `pantauBanyakResi` disusun untuk memproses lima resi sekaligus. Program diuji pada dua skenario: seluruh resi valid dan terdapat satu resi tidak valid (XYZ-999). Kerapian kode diperiksa dengan `dart analyze latihan/modul03_nullsafety.dart latihan/modul03_async.dart`.

4. Kode Program dan Penjelasan

4.1 latihan/modul03_nullsafety.dart

Berkas ini memuat latihan *null safety* dan penanganan *error* pada pencarian resi. Data resi disimpan dalam `Map`, dan hasil pencarian ditampilkan berupa kota tujuan. Pengujian dilakukan pada dua resi, yaitu SLG-001 yang terdaftar dan SLG-999 yang tidak terdaftar.

4.2 Null Safety dan nullable property

Pada Dart, tipe biasa seperti `String` tidak boleh bernilai `null`. Apabila suatu properti memang boleh kosong, tipenya harus ditulis dengan tanda tanya, yaitu `String?`. Properti `catatan` dibuat *nullable* karena tidak setiap kiriman memiliki catatan. Dengan cara ini, kemungkinan nilai kosong dinyatakan secara eksplisit pada tipe data, sehingga *analyzer* dapat memeriksanya sebelum program dijalankan, bukan baru ditemukan ketika program berjalan. Pada kode juga terdapat ekspresi yang memakai operator `??` (baris 35) untuk memberi nilai cadangan bila nilai di sebelah kirinya `null`. Operator ini hanya bermakna selama nilai di sebelah kirinya memang bisa `null`; hal tersebut terbukti pada pengujian di bagian 5.2.

4.3 Error Handling dan ResiTidakDitemukan

`ResiTidakDitemukan` adalah *exception* khusus yang dilempar ketika resi yang dicari tidak ada dalam sistem. Pesannya berbentuk *Resi "SLG-999" tidak ada dalam sistem*. Pemanggilan pencarian ditempatkan pada blok `try`. Klausa `on ResiTidakDitemukan` hanya menangkap jenis *exception* tersebut sehingga penanganannya spesifik, dan pada program ini menghasilkan baris berawalan `[ERROR]`. Blok `finally` selalu dijalankan, baik pencarian berhasil maupun gagal, sehingga pesan *Pencarian untuk "... selesai*. tercetak pada kedua kondisi. Dengan demikian program tidak berhenti mendadak ketika resi tidak ditemukan.

4.4 latihan/modul03_async.dart

Berkas ini memuat tiga bagian yang tampak pada keluaran program: Latihan 3 (*async / await*),

Latihan 4 (*Future.wait*), dan Tugas Praktikum yang menjalankan `pantauBanyakResi` pada Skenario 1 (seluruh resi valid) dan Skenario 2 (ada resi tidak valid).

4.5 Future, async, dan await

Operasi yang membutuhkan waktu tunggu, misalnya mengambil status kiriman dari sistem, tidak langsung menghasilkan nilai. Fungsi seperti ini ditandai `async` dan mengembalikan objek `Future`: fungsi pengambil status mengembalikan `Future<String>`, sedangkan fungsi penghitung ongkos kirim mengembalikan `Future<double>`. Kata kunci `await` digunakan untuk menunggu `Future` selesai dan mengambil nilainya, sehingga program dapat melanjutkan ke baris berikutnya dengan nilai yang sudah siap, seperti pada status *Dalam perjalanan menuju Surabaya* dan ongkos *Rp106250* untuk SLG-002.

4.6 Future.wait

Pada proses sekuensial, setiap `await` menunggu operasi sebelumnya selesai sehingga total waktu merupakan penjumlahan waktu tiap operasi. Pada `Future.wait`, seluruh operasi dimulai bersamaan lalu program menunggu sampai semuanya selesai, sehingga total waktu mendekati waktu operasi yang paling lama. Pengukuran pada Gambar 3 menunjukkan sekitar 3 detik untuk proses sekuensial dan sekitar 2 detik untuk proses paralel, yang konsisten dengan dua operasi berjeda sekitar 1 detik dan 2 detik. Inilah alasan proses paralel lebih cepat pada simulasi jeda yang digunakan: waktu tunggu kedua operasi saling tumpang tindih, bukan saling menunggu.

4.7 pantauBanyakResi

Fungsi `pantauBanyakResi` memproses daftar resi sekaligus. Setiap resi diproses sebagai satu `Future` tersendiri, kemudian seluruhnya ditunggu dengan `Future.wait`. Kesalahan ditangani pada tingkat tiap resi, bukan pada tingkat keseluruhan daftar, sehingga resi yang gagal menghasilkan baris `[GAGAL]` beserta pesan kesalahannya, sedangkan resi yang berhasil menghasilkan baris `[OK]` beserta status kiriman. Setelah seluruh `Future` selesai, program mencetak *Seluruh daftar selesai diproses* dan durasi total.

5. Hasil Eksekusi dan Analisis

5.1 Pengujian Null Safety dan Error Handling

```
PS C:\Users\ACER\Documents\pemrog4\Pertemuan 3\flutter_application_1\silog_app> dart run latihan/modul03_nullsafety.dart

Mencari resi: SLG-001
Kota tujuan: Bandung
[finally] Pencarian untuk "SLG-001" selesai.

Mencari resi: SLG-999
[ERROR] ResiTidakDitemukan: Resi "SLG-999" tidak ada dalam sistem.
[finally] Pencarian untuk "SLG-999" selesai.

=====
```

Gambar 1. Hasil pengujian pencarian resi dengan kondisi resi ditemukan dan resi tidak ditemukan.

Pencarian SLG-001 menghasilkan data kota tujuan **Bandung**. Pencarian SLG-999 memicu *exception* `ResiTidakDitemukan` dengan pesan bahwa resi tersebut tidak ada dalam sistem, dan *exception* ini ditangkap sehingga program tetap berjalan. Blok `finally` tetap dijalankan pada kedua kondisi, terlihat dari pesan `[finally] Pencarian untuk "SLG-001" selesai.` dan `[finally] Pencarian untuk "SLG-999" selesai.`

5.2 Pengujian Error Null Safety

```
PS C:\Users\ACER\Documents\pemrog4\Pertemuan 3\flutter_application_1\silog_app> dart analyze latihan/modul03_nullsafety.dart
Analyzing modul03_nullsafety.dart...

error • modul03_nullsafety.dart:24:10 • The parameter 'catatan' can't have a value of 'null' because of its type, but the
implicit default value is 'null'. Try adding either an explicit non-'null' default value or the 'required'
modifier. • missing_default_value_for_parameter
warning • modul03_nullsafety.dart:35:35 • Dead code. Try removing the code, or fixing the code before it so that it can be
reached. • dead_code
warning • modul03_nullsafety.dart:35:38 • The left operand can't be null, so the right operand is never executed. Try
removing the operator and the right operand. • dead_null_aware_expression

3 issues found.
```

Gambar 2. Hasil pengujian Null Safety ketika properti nullable diubah menjadi non-nullable.

Gambar 2 merupakan hasil **pengujian sementara**, bukan kondisi akhir kode. Properti `String?` `catatan` diubah menjadi `String` `catatan`, lalu dijalankan `dart analyze`. *Analyzer* melaporkan

3 temuan: satu *error* pada `modul03_nullsafety.dart:24:10` (*missing_default_value_for_parameter*), yaitu parameter `catatan` tidak boleh bernilai `null` karena tipenya, padahal nilai bawaannya implisit `null`; serta dua *warning* pada baris 35, yaitu *dead_code* dan *dead_null_aware_expression*, karena operan kiri tidak mungkin `null` sehingga operan kanan tidak akan pernah dijalankan.

Hasil ini menunjukkan bahwa `String?` memungkinkan nilai `null`, sedangkan `String` tidak boleh menerimanya, dan Dart *analyzer* mendeteksi ketidaksesuaian tersebut sebelum program dijalankan.

Setelah pengujian, kode dikembalikan menjadi `String?` `catatan`. **5.3 Pengujian Async/Await**

```
PS C:\Users\ACER\Documents\pemrog4\Pertemuan 3\flutter_application_1\silog_app> dart run latihan/modul03_async.dart
===== LATIHAN 3: ASYNC / AWAIT =====

Mengambil status kiriman SLG-002...
Status kiriman : Dalam perjalanan menuju Surabaya
Ongkos kirim   : Rp106250

=====

===== LATIHAN 4: FUTURE.WAIT =====
Mengambil status & ongkir SLG-001 secara SEKUENSIAL...
Status : Paket telah diterima di gudang Bandung
Ongkir  : Rp18000
Durasi  : 3022 ms (berjalan sekuensial)

Mengambil status & ongkir SLG-001 secara PARALEL...
Status : Paket telah diterima di gudang Bandung
Ongkir  : Rp18000
Durasi  : 2009 ms (berjalan paralel)
=====

===== TUGAS PRAKTIKUM 2 =====

SKENARIO 1 - SELURUH RESI VALID

--- Memproses 5 resi secara paralel ---

[OK] SLG-001 → Paket telah diterima di gudang Bandung
[OK] SLG-002 → Dalam perjalanan menuju Surabaya
[OK] SLG-003 → Paket tiba di hub Makassar, menunggu kurir
[OK] SLG-004 → Sedang dikirim ke alamat tujuan Jayapura
[OK] SLG-005 → Paket telah diterima oleh penerima di Medan

--- Seluruh daftar selesai diproses ---
Durasi total: 2006 ms
```

Gambar 3. Hasil pengujian Async/Await, Future.wait, perbandingan proses sequential dan parallel, serta pemrosesan resi valid.

Pada bagian **LATIHAN 3: ASYNC / AWAIT**, program mengambil status kiriman SLG-002 dan menampilkan **Dalam perjalanan menuju Surabaya** dengan ongkos kirim **Rp106250**. Nilai tersebut dapat tercetak sebagai teks dan angka karena `await` menunggu `Future` selesai terlebih dahulu sebelum hasilnya digunakan.

5.4 Perbandingan Sequential dan Parallel

Pada bagian *LATIHAN 4: FUTURE.WAIT* (Gambar 3), status dan ongkir SLG-001 diambil dengan dua cara. Hasil kedua cara sama, yaitu status *Paket telah diterima di gudang Bandung* dan ongkir *Rp18000*, tetapi durasinya berbeda:

Cara pemrosesan	Durasi	Keterangan
Sekuensial	3022 ms	Operasi dijalankan satu per satu
Paralel (<i>Future.wait</i>)	2009 ms	Operasi dijalankan bersamaan

Proses paralel lebih cepat sekitar 1013 ms ($3022 - 2009$) daripada proses sekuensial. *Future.wait* memungkinkan dua operasi berjalan secara paralel sehingga total waktu hanya ditentukan oleh operasi yang paling lama, sedangkan pada proses sekuensial waktu kedua operasi dijumlahkan.

5.5 Skenario Seluruh Resi Valid

Pada Skenario 1 (Gambar 3), lima resi, yaitu SLG-001 sampai SLG-005, diproses secara paralel dan seluruhnya berstatus `[OK]` dengan status pengiriman masing-masing. Durasi total proses adalah **2006 ms**, hampir sama dengan durasi satu operasi paralel pada Latihan 4 dan tidak berlipat lima kali, yang menunjukkan kelima resi diproses bersamaan.

5.6 Skenario Resi Tidak Valid

```
SKENARIO 2 - ADA RESI TIDAK VALID

--- Memproses 5 resi secara paralel ---

[OK]  SLG-001 → Paket telah diterima di gudang Bandung
[OK]  SLG-002 → Dalam perjalanan menuju Surabaya
[GAGAL] XYZ-999 → FORMAT ERROR: Format resi tidak valid: "XYZ-999". Gunakan format SLG-XXX.
[OK]  SLG-004 → Sedang dikirim ke alamat tujuan Jayapura
[OK]  SLG-005 → Paket telah diterima oleh penerima di Medan

--- Seluruh daftar selesai diproses ---
Durasi total: 2011 ms

===== SELESAI =====
```

Gambar 4. Pengujian pemrosesan banyak resi dengan satu resi tidak valid.

Pada Skenario 2, resi ketiga diganti dengan XYZ-999. Resi ini menghasilkan baris `[GAGAL]` dengan pesan **FORMAT ERROR: Format resi tidak valid: "XYZ-999". Gunakan format SLGXXX**. Resi lainnya (SLG-001, SLG-002, SLG-004, dan SLG-005) tetap diproses dan berstatus `[OK]`. Hal ini menunjukkan bahwa kegagalan satu *Future* tidak menghentikan *Future* lainnya, dan seluruh daftar tetap selesai diproses dengan durasi total 2011 ms.

5.7 Pengujian Tanpa Await

```
PS C:\Users\ACER\Documents\pemrog4\Pertemuan 3\flutter_application_1\silog_app> dart run latihan/modul03_async.dart
===== LATIHAN 3: ASYNC / AWAIT =====

Mengambil status kiriman SLG-002...
Status kiriman : Instance of 'Future<String>'
Ongkos kirim   : Instance of 'Future<double>'

=====

===== LATIHAN 4: FUTURE.WAIT =====
Mengambil status & ongkir SLG-001 secara SEKUENSIAL...
Status        : Paket telah diterima di gudang Bandung
Ongkir        : Rp18000
Durasi        : 3018 ms (berjalan sekuensial)

Mengambil status & ongkir SLG-001 secara PARALEL...
Status        : Paket telah diterima di gudang Bandung
Ongkir        : Rp18000
Durasi        : 2008 ms (berjalan paralel)

=====
```

Gambar 5. Hasil pengujian ketika await dihilangkan pada pemanggilan fungsi asynchronous.

Gambar 5 merupakan hasil **eksperimen sementara** untuk memahami Future dan await, bukan error program akhir. Ketika await dihilangkan, keluaran menjadi Status kiriman :

Instance of 'Future<String>' dan Ongkos kirim : Instance of 'Future<double>'.

Hal ini terjadi karena fungsi async selalu mengembalikan Future; tanpa await, nilai di dalam Future belum diambil sehingga yang tercetak adalah objek Future-nya, bukan isinya. Setelah pengujian, await dikembalikan ke kode normal.

5.8 Hasil Dart Analyze

```
PS C:\Users\ACER\Documents\pemrog4\Pertemuan 3\flutter_application_1\silog_app> dart analyze latihan/modul03_nullsafety.dart latihan/modul03_async.dart
Analyzing modul03_nullsafety.dart, modul03_async.dart...
No issues found!
```

Gambar 6. Hasil dart analyze pada dua file Modul 03.

Perintah `dart analyze latihan/modul03_nullsafety.dart`

`latihan/modul03_async.dart` menganalisis kedua berkas dan melaporkan *No issues found!*.

Dengan demikian `latihan/modul03_nullsafety.dart` dan `latihan/modul03_async.dart` tidak memiliki temuan *error* maupun *warning*. Pemeriksaan ini mencakup kedua berkas Modul 03 tersebut.

6. Jawaban Tugas Praktikum

Tugas Praktikum Modul 03 diimplementasikan pada `latihan/modul03_async.dart` melalui fungsi `pantauBanyakResi`, yang memantau lima resi sekaligus dan menampilkan hasil tiap resi sebagai [OK] atau [GAGAL]. Cara kerja program adalah sebagai berikut:

- **Membaca data resi dari Map.** Data status kiriman disimpan dalam Map dengan nomor resi sebagai kunci. Untuk tiap nomor resi pada daftar yang dipantau, program mengambil statusnya dari data tersebut.
- **Menangani resi yang tidak dapat diproses.** Setiap resi diproses dalam penanganan *error* tersendiri. Resi yang tidak memenuhi syarat, misalnya XYZ-999 yang formatnya tidak sesuai pola SLG-XXX (Gambar 4), tidak membuat program berhenti, tetapi dicatat sebagai [GAGAL] beserta pesan kesalahannya. Resi yang tidak terdaftar pada data diperlakukan sebagai kegagalan individual dengan mekanisme yang sama.
- **Memproses banyak resi secara concurrent.** Tiap resi dijalankan sebagai Future dan seluruhnya ditunggu bersama dengan `Future.wait`. Pada Skenario 1, lima resi selesai dalam 2006 ms, sehingga waktunya tidak bertambah menjadi kelipatan lima.

- **Tetap melanjutkan proses ketika salah satu resi gagal.** Karena kesalahan ditangani pada tiap `Future`, kegagalan XYZ-999 pada Skenario 2 tidak menghentikan SLG-001, SLG002, SLG-004, dan SLG-005, yang tetap selesai diproses dalam 2011 ms.
- **Menghasilkan daftar [OK] dan [GAGAL].** Setelah semua `Future` selesai, hasil dicetak berurutan sesuai daftar resi: baris [OK] memuat nomor resi dan status kiriman, sedangkan baris [GAGAL] memuat nomor resi dan pesan kesalahan, diikuti pesan *Seluruh daftar selesai diproses* dan durasi total.

Tabel skenario uji

No	Skenario	Input	Output yang diharapkan	Output yang diperoleh	Status
1	Null Safety normal	<code>dart run latihan/modul03_nullsafety.dart</code> dengan String? catatan	Program berjalan tanpa error	Program berjalan; hasil seperti pada Gambar 1	Sesuai
2	Null Safety ketika ? dihapus sementara	<code>String</code> catatan, lalu <code>dart analyze</code> pada <code>modul03_nullsafety.dart</code>	Analyzer mendeteksi pelanggaran null safety	3 issues: 1 error (24:10, <code>missing_default_value_for_parameter</code>) dan 2 warning (35:35 <code>dead_code</code> , 35:38 <code>dead_null_aware_expression</code>)	Sesuai
3	SLG-001 valid	Cari resi SLG-001	Data ditemukan, kota tujuan Bandung, finally berjalan	Kota tujuan: Bandung; [finally] untuk SLG-001 tercetak	Sesuai
4	SLG-999 tidak terdaftar	Cari resi SLG-999	<code>ResiTidakDitemukan</code> ditangkap, finally tetap berjalan	[ERROR] <code>ResiTidakDitemukan: Resi "SLG-999" tidak ada dalam sistem.; [finally] untuk SLG-999 tercetak</code>	Sesuai
5	Async/Await SLG-002	Ambil status dan ongkos kirim SLG002 dengan <code>await</code>	Status dan ongkos kirim tercetak sebagai nilai	Dalam perjalanan menuju Surabaya; Rp106250	Sesuai
6	Sequential vs <code>Future.wait</code>	Status dan ongkir SLG-001	Proses paralel lebih cepat daripada sekuensial	Sekuensial 3022 ms; paralel 2009 ms	Sesuai
7	Lima resi valid	SLG-001 s.d. SLG005	Seluruh resi [OK]	Lima resi [OK]; durasi total 2006 ms	Sesuai
8	Lima resi dengan satu resi invalid	SLG-001, SLG-002, XYZ-999, SLG-004, SLG-005	XYZ-999 [GAGAL], resi lain tetap [OK]	XYZ-999 [GAGAL] (FORMAT ERROR); empat resi lain [OK]; durasi total 2011 ms	Sesuai
9	Tanpa <code>await</code>	<code>await</code> dihapus sementara pada pemanggilan fungsi <code>async</code>	Yang tercetak adalah objek <code>Future</code>	Instance of <code>'Future<String>'</code> dan Instance of <code>'Future<double>'</code>	Sesuai
10	<code>dart analyze</code> dua file Modul 03	<code>dart analyze latihan/modul03_nullsafety.dart latihan/modul03_async.dart</code>	Tidak ada temuan	No issues found!	Sesuai

7. Kesimpulan dan Kendala

Kesimpulan. Praktikum Modul 03 berhasil dilaksanakan dan seluruh skenario uji menghasilkan keluaran sesuai yang diharapkan. Tipe `nullable String?` membuat kemungkinan nilai kosong dinyatakan secara eksplisit, dan pengujian dengan `String` membuktikan bahwa *analyzer* Dart mendeteksi pelanggaran tersebut. *Exception* khusus `ResiTidakDitemukan` bersama `try / on / finally` membuat program

tetap berjalan ketika resi tidak ditemukan. Kata kunci `await` diperlukan untuk mengambil nilai dari `Future`, terbukti dari keluaran `Instance of 'Future<...>'` ketika `await` dihilangkan. `Future.wait` terbukti mempersingkat waktu proses dari 3022 ms menjadi 2009 ms pada simulasi jeda yang digunakan, dan pada `pantauBanyakResi` kegagalan satu resi tidak menghentikan resi lainnya. Pemeriksaan `dart analyze` pada kedua berkas Modul 03 menghasilkan *No issues found!*.

Kendala. Pada kode akhir tidak ditemukan *error* maupun *warning*. *Error* dan *warning* pada Gambar 2 muncul hanya karena pengujian sementara dan tidak ada lagi setelah kode dikembalikan menjadi `String?` catatan, sebagaimana ditunjukkan Gambar 6. Hal yang memerlukan ketelitian adalah penulisan `await` pada setiap pemanggilan fungsi `async` agar nilai yang digunakan adalah hasil `Future`, bukan objek `Future`-nya, serta penempatan penanganan *error* pada tiap resi agar kegagalan satu resi tidak menghentikan proses resi lainnya.

8. Tautan Repositori dan Riwayat Commit

Repositori GitHub: <https://github.com/Nogi3201/silog-714240045>

Cabang: <https://github.com/Nogi3201/silog-714240045/tree/main/Pertemuan%203>

Riwayat commit: Modul 3: Null Safety, Error Handling, dan Async